



Sprint 3: Modeling Baseline Entrenamiento, Evaluación y Comparación

Sprint Backlog, Roles y Definition of Done

Pedro Shiguihara, pshiguihara@pucp.edu.pe

17 de abril de 2026

Agenda

1. Contexto: de la preparación de datos al modelado
2. Metodología: CRISP-DM y Scrum aplicados al Sprint 3
3. Product Backlog: ítems del Sprint 3
4. Sprint 3: Alcance y Sprint Planning
5. Roles del equipo (5 personas)
6. Detalle de tareas por rol
7. Entregables del Sprint 3
8. Criterios de aceptación y Definition of Done

Contexto del Sprint 3

Del Sprint 2 (preparar) al Sprint 3 (modelar)

🎯 Objetivo del Sprint 3

Entrenar **modelos baseline** de clasificación sobre el dataset preparado en el Sprint 2, evaluarlos con métricas apropiadas y **comparar** su rendimiento para seleccionar los mejores candidatos que serán optimizados en el Sprint 4.

➔ Insumos del Sprint 2

- 05_data_cleaning.ipynb (limpieza documentada)
- 06_feature_eng.ipynb (features derivadas)
- 07_class_balance.ipynb (balanceo de clases)
- 08_pipeline.ipynb (pipeline integrado)
- src/preprocessing.py (código reutilizable)
- models/preproc.pkl (pipeline persistido)
- Dataset procesado en data/processed/

🔧 Herramientas Clave

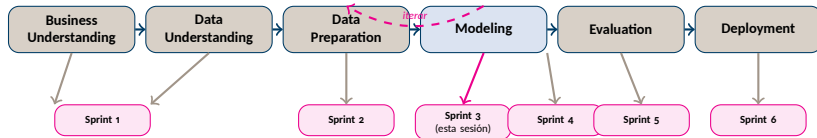
- Python 3.10+
- scikit-learn (modelos, métricas, cross-validation)
- matplotlib / seaborn (visualizar resultados)
- pandas (tablas comparativas)
- joblib (persistir modelos)
- DVC (versionar modelos)
- Git + GitHub Projects

⚠ Clave

Este sprint entrena modelos **con parámetros por defecto** (baselines). La optimización de hiperparámetros se hará en el Sprint 4.

Metodología: CRISP-DM dentro de Scrum

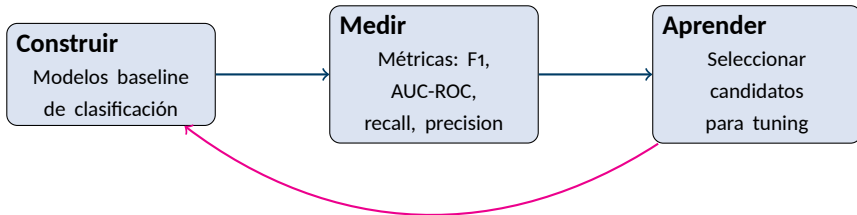
Mapeo de fases CRISP-DM a 6 Sprints — foco en Sprint 3



- **Sprint 1:** Business & Data Understanding.
- **Sprint 2:** Data Preparation.
- **Sprint 3:** Modeling baseline (*esta sesión*).
- **Sprint 4:** Modeling avanzado + tuning.
- **Sprint 5:** Evaluation + Business Value.
- **Sprint 6:** Deployment en AWS (MVP).

Ciclo Lean Startup aplicado al Sprint 3

Construir → Medir → Aprender



💡 Filosofía del Sprint 3

El objetivo **no es** obtener el mejor modelo posible, sino establecer **baselines** con parámetros por defecto que sirvan como punto de referencia. Un baseline robusto permite medir cuánto mejora la optimización en el Sprint 4 y evita gastar tiempo tuneando modelos que no son competitivos.

Product Backlog Completo

Historias de usuario para todo el proyecto (6 sprints)

ID	Historia de Usuario	Fase CRISP-DM	Sprint
PB-01	Definir el problema de negocio y objetivos	Business Underst.	1
PB-02	Identificar criterios de éxito del modelo	Business Underst.	1
PB-03	Cargar y explorar el dataset inicial	Data Understanding	1
PB-04	Analizar calidad de datos (nulos, tipos, duplicados)	Data Understanding	1
PB-05	Generar estadísticas descriptivas y visualizaciones	Data Understanding	1
PB-06	Construir prototipo interactivo de exploración	Data Understanding	1
PB-07	Limpiar y transformar variables	Data Preparation	2
PB-08	Realizar feature engineering	Data Preparation	2
PB-09	Balancear clases si es necesario	Data Preparation	2
PB-10	Entrenar modelos baseline de clasificación	Modeling	3
PB-11	Evaluar modelos baseline con métricas	Modeling	3
PB-12	Comparar modelos y seleccionar candidatos	Modeling	3
PB-13	Optimizar hiperparámetros del mejor modelo	Modeling	4
PB-14	Aplicar técnicas avanzadas (ensambles, stacking)	Modeling	4
PB-15	Validar modelo final (cross-validation, test set)	Modeling	4
PB-16	Evaluar Business Value del modelo seleccionado	Evaluation	5
PB-17	Construir dashboard interactivo de resultados	Evaluation	5
PB-18	Documentar hallazgos y recomendaciones de negocio	Evaluation	5
PB-19	Empaquetar modelo como API (Flask/FastAPI)	Deployment	6
PB-20	Desplegar prototipo MVP en AWS (EC2/Lambda)	Deployment	6
PB-21	Integrar modelo con dashboard desplegado	Deployment	6

→ Las filas resaltadas en azul corresponden al **Sprint 3** (esta iteración): PB-10, PB-11 y PB-12.

Sprint 3: Sprint Planning

Alcance: Modeling baseline — entrenar, evaluar y comparar

Meta del Sprint

Entrenar al menos **4-5 modelos baseline** de clasificación con parámetros por defecto, evaluarlos con métricas robustas (F1, AUC-ROC, recall) usando **cross-validation**, y producir una **tabla comparativa** que justifique qué modelos pasan al Sprint 4 para tuning.

Duración

- **1 semana** de trabajo colaborativo
- Daily stand-ups (async vía GitHub Issues)
- Sprint Review al final de la semana

Sprint Backlog (3 ítems grandes)

- PB-10** Entrenar modelos baseline (Logistic Regression, Decision Tree, Random Forest, SVM, KNN, etc.).
- PB-11** Evaluar cada modelo con métricas de clasificación y cross-validation.
- PB-12** Comparar modelos, seleccionar top candidatos y documentar decisiones.

Gestión

- Mismo repositorio GitHub de Sprints 1 y 2
- Cada sub-tarea = 1 Issue en el board Kanban
- Ramas por feature: `feat/PB-XX`

Roles del Equipo: 5 Personas

Roles adaptados al Sprint 3 (Modeling Baseline)



Project Manager
Coordina, gestiona el
Kanban y revisa entregables



Baseline Trainer
Entrena modelos baseline con parámetros por defecto (PB-10)



Metrics Evaluator
Evalúa métricas y cross-validation (PB-11)



Model Comparator
Compara modelos y selecciona candidatos (PB-12)



Experiment Tracker
Registra experimentos, versiona modelos y documenta

Nota importante

Los roles **rotan** nuevamente respecto al Sprint 2: cada miembro del equipo asume un rol distinto para ganar experiencia en la fase de Modeling. Todos colaboran y revisan el trabajo de los demás vía **Pull Requests**.

Rol 1: Project Manager

 Coordinación, planificación y seguimiento

Tareas del Sprint 3

1. Crear los Issues del Sprint 3 (PB-10, PB-11, PB-12) y sub-issues por tarea.
2. Reorganizar el board Kanban (cerrar Sprint 2, abrir Sprint 3).
3. Asignar responsables y revisores cruzados a cada Issue.
4. Verificar que el pipeline del Sprint 2 funciona correctamente antes de iniciar el modelado.
5. Coordinar la secuencia: entrenamiento → evaluación → comparación.
6. Preparar el Sprint Review: demo de la tabla comparativa y selección de candidatos.

Estructura del Repo (Sprint 3)

```
proyecto/  
+-- data/  
|   +-- raw/  
|   +-- interim/  
|   +-- processed/  
+-- notebooks/  
|   +-- 09_baseline_models.ipynb  
|   +-- 10_evaluation.ipynb  
|   +-- 11_model_comparison.ipynb  
+-- src/  
|   +-- preprocessing.py  
|   +-- models.py  
+-- models/  
|   +-- preproc.pkl  
|   +-- baseline_*.pkl  
+-- environment.yml
```

Rol 2: Baseline Trainer

Entrenamiento de modelos baseline (PB-10)

Tareas del Sprint 3

1. Cargar el dataset procesado y el pipeline del Sprint 2.
2. Entrenar al menos **5 modelos baseline** con parámetros por defecto:
 - Logistic Regression
 - Decision Tree
 - Random Forest
 - Support Vector Machine (SVM)
 - K-Nearest Neighbors (KNN)
 - (Opcional) Naive Bayes, Gradient Boosting
3. Usar el pipeline de preprocesamiento con cada modelo dentro de un Pipeline completo.
4. Persistir cada modelo entrenado con joblib en models/.
5. Documentar cada modelo: qué es, cómo funciona y por qué se incluye.

Ejemplo

```
from sklearn.pipeline import Pipeline
from sklearn.linear_model import (
    LogisticRegression)
from sklearn.ensemble import (
    RandomForestClassifier)
from sklearn.svm import SVC
from sklearn.neighbors import (
    KNeighborsClassifier)
from sklearn.tree import (
    DecisionTreeClassifier)
import joblib

# Cargar pipeline de preprocesamiento
preproc = joblib.load(
    'models/preproc.pkl')

# Modelo completo = preproc + clasificador
pipe_lr = Pipeline([
    ('preproc', preproc),
    ('clf', LogisticRegression(
        random_state=42)),
])

pipe_lr.fit(X_train, y_train)
joblib.dump(pipe_lr,
    'models/baseline_lr.pkl')
```

Rol 3: Metrics Evaluator

 Evaluación con métricas de clasificación (PB-11)

Tareas del Sprint 3

1. Definir el **conjunto de métricas** a evaluar según el problema de negocio:
 - **Accuracy**: proporción de aciertos totales.
 - **Precision**: de los predichos positivos, cuántos son correctos.
 - **Recall**: de los positivos reales, cuántos se detectaron.
 - **F1-Score**: media armónica de precision y recall.
 - **AUC-ROC**: capacidad de discriminación del modelo.
2. Aplicar **cross-validation** (5-fold stratified) para obtener métricas robustas.
3. Generar `classification_report` y `confusion_matrix` para cada modelo.
4. Visualizar curvas ROC y Precision-Recall.

Ejemplo

```
from sklearn.model_selection import (
    cross_validate,
    StratifiedKFold)
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    roc_auc_score,
    make_scorer)

cv = StratifiedKFold(
    n_splits=5, shuffle=True,
    random_state=42)

scoring = {
    'accuracy': 'accuracy',
    'f1': 'f1',
    'precision': 'precision',
    'recall': 'recall',
    'roc_auc': 'roc_auc',
}

results = cross_validate(
    pipe_lr, X_train, y_train,
    cv=cv, scoring=scoring,
    return_train_score=True)

print(f"F1 (CV): "
      f"{results['test_f1'].mean():.3f} "
      f"+/- {results['test_f1'].std():.3f}")
```

Rol 3: Metrics Evaluator — Visualizaciones Clave

 Curvas ROC y Matriz de Confusión

</> Curva ROC

```
from sklearn.metrics import (
    RocCurveDisplay)
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(7, 5))

for name, pipe in models.items():
    RocCurveDisplay.from_estimator(
        pipe, X_test, y_test,
        name=name, ax=ax)

ax.plot([0, 1], [0, 1], 'k--',
        label='Azar (0.5)')
ax.set_title('Curvas ROC - Baselines')
ax.legend(loc='lower right')
plt.tight_layout()
plt.show()
```

</> Matriz de Confusión

```
from sklearn.metrics import (
    ConfusionMatrixDisplay)

fig, axes = plt.subplots(
    2, 3, figsize=(14, 8))

for ax, (name, pipe) in zip(
    axes.ravel(),
    models.items()):
    ConfusionMatrixDisplay.from_estimator(
        pipe, X_test, y_test,
        ax=ax, cmap='Blues',
        colorbar=False)
    ax.set_title(name, fontsize=10)

plt.suptitle(
    'Matrices de Confusión - Baselines')
plt.tight_layout()
plt.show()
```

Rol 4: Model Comparator

 Comparación y selección de candidatos (PB-12)

Tareas del Sprint 3

1. Consolidar resultados de todos los baselines en una **tabla comparativa** con métricas de cross-validation.
2. **Ranear** modelos por la métrica principal definida en el Sprint 1 (e.g., F1, AUC-ROC).
3. Analizar **trade-offs**: precision vs. recall, velocidad de entrenamiento, interpretabilidad.
4. Seleccionar los **top 2-3 candidatos** que pasarán al Sprint 4 para tuning.
5. Documentar la **justificación** de la selección: por qué se descarta cada modelo eliminado.
6. Generar visualizaciones comparativas (barras agrupadas, radar charts).

Tabla Comparativa

```
import pandas as pd

results_list = []
for name, pipe in models.items():
    scores = cross_validate(
        pipe, X_train, y_train,
        cv=cv, scoring=scoring)
    results_list.append({
        'Modelo': name,
        'Accuracy': scores[
            'test_accuracy'].mean(),
        'F1': scores[
            'test_f1'].mean(),
        'AUC-ROC': scores[
            'test_roc_auc'].mean(),
        'Recall': scores[
            'test_recall'].mean(),
        'Precision': scores[
            'test_precision'].mean(),
    })

df_results = pd.DataFrame(results_list)
df_results.sort_values(
    'F1', ascending=False, inplace=True)
df_results.style.highlight_max(
    axis=0, color='lightgreen')
```

Rol 5: Experiment Tracker

 Registro de experimentos y versionado de modelos

Tareas del Sprint 3

1. Encapsular la lógica de entrenamiento y evaluación en funciones reutilizables en `src/models.py`.
2. Persistir todos los modelos entrenados con `joblib` y versionar con **DVC**.
3. Crear un **registro de experimentos**: tabla/CSV con modelo, parámetros, métricas y fecha.
4. Validar que cada modelo se puede cargar y predecir en un notebook independiente.
5. Documentar el flujo completo: desde datos crudos hasta predicción, pasando por pipeline + modelo.
6. Preparar la estructura para el Sprint 4 (tuning).

`src/models.py`

```
import joblib
import pandas as pd
from sklearn.model_selection import (
    cross_validate)

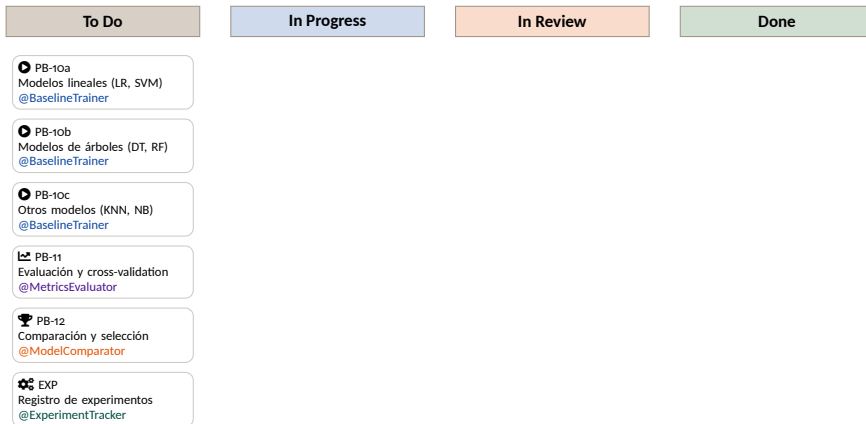
def train_baseline(pipe, X, y, name):
    """Entrena y persiste un baseline."""
    pipe.fit(X, y)
    joblib.dump(pipe,
        f'models/baseline_{name}.pkl')
    return pipe

def evaluate_model(pipe, X, y, cv,
    scoring):
    """Evalua con cross-validation."""
    return cross_validate(
        pipe, X, y, cv=cv,
        scoring=scoring,
        return_train_score=True)

def log_experiment(name, params,
    metrics, path):
    """Registra un experimento."""
    row = {'model': name,
        **params, **metrics}
    df = pd.DataFrame([row])
    df.to_csv(path,
        mode='a', header=False)
```

Tablero Kanban en GitHub Projects

Visualización del flujo de trabajo del Sprint 3



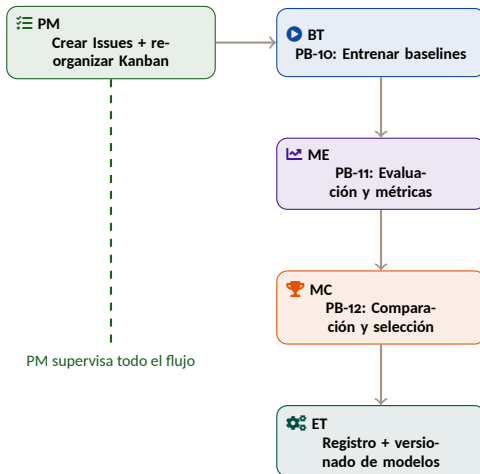
Flujo de Trabajo en el Kanban

Flujo

Cada Issue se mueve: **To Do** → se crea rama feat/PB-XX → **In Progress** → se abre Pull Request → **In Review** → se aprueba y mergea → **Done**. El Experiment Tracker integra al final, una vez los modelos están entrenados y evaluados.

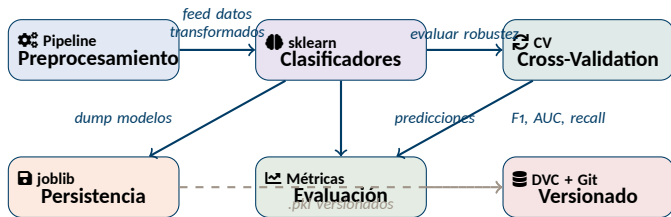
Dependencias entre Tareas

Orden sugerido de ejecución en el Sprint 3



Herramientas del Sprint 3: ¿Cómo encajan juntas?

Visión integrada para Modeling Baseline



💡 Idea clave

El Pipeline del Sprint 2 se reutiliza como primera etapa de cada modelo. Cada baseline es un Pipeline (preproc, clf) que va de datos crudos a predicción en un solo objeto.

Tip 1: Pipeline completo = Preprocesamiento + Modelo

⚙️ Un solo objeto que transforma y predice

✖ Mal: pasos sueltos

```
# MAL: preprocesar por separado
X_train_t = preproc.fit_transform(X_train)
X_test_t = preproc.transform(X_test)
```

```
clf = RandomForestClassifier()
clf.fit(X_train_t, y_train)
y_pred = clf.predict(X_test_t)
```

```
# Problema: en cross_validate
# no se puede pasar preproc + clf
# como un solo objeto
```

✔ Bien: Pipeline integrado

```
# BIEN: todo en un Pipeline
from sklearn.pipeline import Pipeline
```

```
pipe = Pipeline([
    ('preproc', preproc),
    ('clf', RandomForestClassifier(
        random_state=42)),
])
```

```
# Cross-validation lo maneja todo:
# fit(preproc) + fit(clf) en train
# transform(preproc) + predict(clf) en val
scores = cross_validate(
    pipe, X_train, y_train, cv=cv)
```

⚠ Regla de oro

Cada baseline debe ser un Pipeline completo que acepte datos **crudos** (sin preprocesar). Así, `cross_validate` aplica el preprocesamiento correctamente en cada fold, evitando data leakage.

Tip 2: Elegir la métrica correcta

🕒 No todos los problemas se evalúan igual

¿Cuándo usar qué?

- **Accuracy:** solo si las clases están balanceadas.
- **F1-Score:** cuando importan tanto precision como recall (e.g., detección de fraude con costo asimétrico).
- **Recall:** cuando el costo de un falso negativo es alto (e.g., diagnóstico médico).
- **Precision:** cuando el costo de un falso positivo es alto (e.g., spam filter).
- **AUC-ROC:** para evaluar la capacidad de discriminación general del modelo.

💡 Conectar con el Sprint 1

- El **Business Analyst** definió los criterios de éxito en `01_business.ipynb`.
- La **métrica principal** del proyecto debe ser coherente con esos criterios.
- Ejemplo: si el negocio prioriza *no perder clientes reales*, la métrica principal es **recall**.
- Documentar esta decisión en el notebook de evaluación.

⚠️ Cuidado

Accuracy con clases desbalanceadas es engañosa: un modelo que predice siempre la clase mayoritaria puede tener 90 % accuracy pero 0 % recall en la clase minoritaria.

Tip 3: Cross-Validation Stratified

 Evaluación robusta que respeta la distribución de clases

¿Por qué Stratified?

- StratifiedKFold mantiene la proporción de clases en cada fold.
- Evita que un fold tenga 0 ejemplos de la clase minoritaria.
- Produce estimaciones más estables de las métricas.
- Es el estándar para problemas de clasificación.



Tip

Usar `random_state=42` en todos los modelos y en el CV para garantizar **reproducibilidad**.

Ejemplo

```
from sklearn.model_selection import (
    StratifiedKFold,
    cross_validate)

cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42)

scoring = ['accuracy', 'f1',
           'precision', 'recall', 'roc_auc']

results = cross_validate(
    pipe, X_train, y_train,
    cv=cv,
    scoring=scoring,
    return_train_score=True)

# Detectar overfitting:
# si train >> test, el modelo
# memoriza en lugar de generalizar
```

Tip 4: Detectar Overfitting y Underfitting

 Las señales de alerta en los resultados de cross-validation

Overfitting

- **Señal:** métricas en `train` muy altas, en `test` mucho más bajas.
- **Ejemplo:** `F1 train` = 0.99, `F1 test` = 0.65.
- **Modelos propensos:** Decision Tree (sin podar), KNN con k pequeño.
- **Solución** (Sprint 4): regularización, poda, más datos, menos features.

Underfitting

- **Señal:** métricas bajas tanto en `train` como en `test`.
- **Ejemplo:** `F1 train` = 0.55, `F1 test` = 0.52.
- **Modelos propensos:** Logistic Regression en datos no lineales, Naive Bayes con features correlacionadas.
- **Solución** (Sprint 4): modelos más complejos, más features, feature engineering.

Regla práctica

En el Sprint 3 **solo observamos** estos patrones y los documentamos. La corrección (tuning, regularización, ensambles) se realiza en el Sprint 4. El baseline es el punto de partida, no el punto final.

Tip 5: Registro de Experimentos

 Mantener un log de cada modelo entrenado

? ¿Por qué registrar?

- Permite **reproducir** cualquier resultado.
- Facilita la **comparación** histórica entre sprints.
- Documenta qué se probó y qué no funcionó.
- Es la base para el Sprint 4 (tuning) y Sprint 5 (evaluación de Business Value).

Formato sugerido: un CSV o DataFrame con columnas: modelo, parámetros, métricas (F1, AUC, etc.), fecha, notas.

</> Ejemplo de registro

```
# experiments_log.csv
experiments = pd.DataFrame({
    'model': ['LR', 'DT', 'RF',
             'SVM', 'KNN'],
    'params': ['default'] * 5,
    'f1_cv_mean': [0.78, 0.71, 0.83,
                  0.80, 0.72],
    'auc_cv_mean': [0.85, 0.68, 0.90,
                  0.87, 0.74],
    'recall_cv_mean': [0.75, 0.73,
                      0.81, 0.77, 0.69],
    'selected': [False, False, True,
                True, False],
    'notes': [
        'buen baseline lineal',
        'overfitting',
        'mejor baseline',
        'lento pero preciso',
        'sensible a escala'],
})
experiments.to_csv(
    'models/experiments_log.csv')
```

Errores comunes en Modeling Baseline

 Qué evitar y cómo resolverlo

	Error común	Solución
⊘	Evaluar solo con accuracy en datos desbalanceados	Usar F1, AUC-ROC, recall y precision como métricas complementarias
⊘	Evaluar en train set (sin cross-validation)	Siempre usar <code>cross_validate</code> con <code>StratifiedKFold</code>
⊘	Tunear hiperparámetros en el Sprint 3	En este sprint solo baselines con parámetros por defecto
⊘	No incluir el preprocesamiento en el Pipeline del modelo	El Pipeline completo debe ir de datos crudos a predicción
⊘	Descartar modelos sin justificar	Documentar por qué cada modelo se descarta (métricas, overfitting, etc.)
⊘	Evaluar en el test set durante la exploración	El test set se usa una sola vez al final para la evaluación definitiva

Entregables del Sprint 3

Lo que cada rol debe entregar al final de la semana

Rol	Entregable	Formato
PM	Issues del Sprint 3 cerrados, board Kanban actualizado, Sprint Review preparado	GitHub Project
BT	Notebook con entrenamiento de 5+ baselines documentados, modelos persistidos en <code>models/</code>	<code>09_baseline_models.ipynb</code>
ME	Notebook con evaluación completa: cross-validation, classification reports, curvas ROC, matrices de confusión	<code>10_evaluation.ipynb</code>
MC	Notebook con tabla comparativa, ranking, selección de top candidatos y justificación	<code>11_model_comparison.ipynb</code>
ET	Módulo <code>src/models.py</code> con funciones reutilizables, registro de experimentos y modelos versionados	<code>src/models.py + experiments_log.csv</code>

Definition of Done (DoD)

Criterios que cada tarea debe cumplir para considerarse terminada

✓ Criterios Generales

- ☐ Código en rama `feat/PB-XX` con Pull Request aprobado y mergeado a `main`.
- ☐ Notebook ejecutable de principio a fin sin errores (*Restart & Run All*).
- ☐ Celdas markdown documentando **cada decisión** de modelado.
- ☐ El entorno se reproduce con `conda env create -f environment.yml`.
- ☐ Issue correspondiente movido a "Done" en el board.

★ Criterios específicos del Sprint 3

- ☐ Al menos 5 modelos baseline entrenados con parámetros por defecto.
- ☐ Cross-validation Stratified (5-fold) aplicada a todos los modelos.
- ☐ Tabla comparativa con métricas: Accuracy, F1, AUC-ROC, Precision, Recall.
- ☐ Top 2-3 candidatos seleccionados con justificación documentada.
- ☐ Modelos persistidos con `joblib` y versionados con DVC.
- ☐ Registro de experimentos en CSV.

Visión General: 6 Sprints hasta el Despliegue

Del entendimiento del negocio al MVP desplegado en AWS

Sprint	Fase CRISP-DM	Objetivo principal
1	Business + Data Underst.	Comprender el problema y explorar los datos
2	Data Preparation	Limpiar, transformar y crear features
3	Modeling	Entrenar modelos baseline y comparar
4	Modeling	Optimizar hiperparámetros y modelos avanzados
5	Evaluation	Evaluar Business Value y documentar resultados
6	Deployment	Desplegar el mejor modelo como MVP en AWS

Meta del Sprint 3

Entregar una **tabla comparativa de baselines** con métricas de cross-validation y una **selección justificada** de los 2-3 mejores candidatos, de modo que el Sprint 4 pueda enfocarse en optimizar hiperparámetros sin tener que explorar modelos desde cero.

Resumen del Sprint 3

❓ ¿Qué?

Modeling Baseline: entrenar, evaluar y comparar clasificadores

👥 ¿Quién?

5 roles: PM, BT, ME, MC, ET

⚙️ ¿Cómo?

scikit-learn + cross-validation + DVC



Entregable

3 Notebooks + `src/models.py`
+ modelos versionados
+ log de experimentos

▶▶ Siguiendo

Sprint 4: Tuning +
modelos avanzados

! Recordar

- El Sprint 3 entrena **solo baselines** (parámetros por defecto). La optimización es en el Sprint 4.
- Siempre evaluar con **cross-validation stratified**, no solo en train/test.
- La métrica principal debe estar alineada con los **criterios de éxito del negocio** (Sprint 1).
- Documentar qué modelos se descartan y **por qué**: esto ahorra tiempo en sprints futuros.

Checklist de Inicio

Pasos inmediatos para comenzar el Sprint 3

1. **PM:** Cerrar el Sprint 2 en el board y crear los Issues del Sprint 3 (PB-10, PB-11, PB-12 y sub-issues).
2. **PM:** Reasignar roles para que cada miembro experimente la fase de Modeling.
3. **Todos:** `git pull && dvc pull` para sincronizar el estado del Sprint 2.
4. **Todos:** Verificar que el pipeline del Sprint 2 funciona: `preproc = joblib.load('models/preproc.pkl')`.
5. **BT:** Iniciar `09_baseline_models.ipynb` en rama `feat/PB-10`, entrenar 5+ modelos baseline.
6. **ME:** Una vez entrenados los modelos, evaluar con cross-validation en `10_evaluation.ipynb` (rama `feat/PB-11`).
7. **MC:** Tras la evaluación, comparar y seleccionar candidatos en `11_model_comparison.ipynb` (rama `feat/PB-12`).

Checklist de Inicio (cont.)

Pasos inmediatos para comenzar el Sprint 3

8. **ET:** Encapsular funciones en `src/models.py`, versionar modelos con DVC y crear el registro de experimentos (rama `feat/EXP`).
9. **Todos:** Sprint Review — presentar la tabla comparativa y los candidatos seleccionados para el Sprint 4.



Diplomado de Especialización en **Data Analytics**